Taller Práctico de Repaso y Consolidación en C++: Recursividad, Búsqueda en Memoria y Punteros

Informe Técnico de Laboratorio de Programación Actividad N.° 6

Autores:

- Ferrari, Ulises (Investigación, Redacción Principal y Formato APA v7)
- Del Pino, Lucas (Programación Backend y Pruebas de Compilación C++)
- Mattia, Federico (Gestión de Repositorio Dual-Remote GitHub/GitLab)
- Orué, Máximo (Líder de Proyecto, Control de Calidad y Traza de Memoria)

Institución: Escuela de Educación Secundaria Técnica N.º 1 "Eduardo Ader" (Vicente López)

Curso: 5.° Año 3.º División (Grupos A-B)

Materia: Laboratorio de Programación

Docente: Prof. Mansilla Muñoz, York Elías

Fecha de Entrega: 1 de octubre de 2026

Resumen

El presente informe técnico expone la resolución analítica, conceptual y práctica correspondiente a la Actividad N.° 6 del espacio curricular Laboratorio de Programación. El trabajo se centra en la consolidación de tres pilares fundamentales del desarrollo de bajo nivel en lenguaje C++: la gestión de la pila de memoria (*Stack*) mediante funciones recursivas con caso base garantizado, la búsqueda secuencial optimizada de complejidad $O(n)$ en arreglos contiguos y la manipulación directa de celdas de memoria RAM a través de punteros e intercambio físico por dirección (EEST N.º 1 "Eduardo Ader", 2026). Asimismo, se documenta la estrategia de resiliencia y control de versiones *Dual-Remote* (GitHub/GitLab) articulada con el proyecto integrador anual *TeKnical May Cry (TmC)*.

Palabras clave: Recursividad, Caso Base, Stack Overflow, Punteros, Desreferenciación, Búsqueda Secuencial, Dual-Remote, C++, APA v7.

1. Introducción

En la arquitectura de software desarrollada en lenguajes de medio y bajo nivel como C++, el control riguroso de los recursos físicos del sistema es un factor determinante para prevenir fallos críticos de ejecución. Durante la interacción con hardware o la ejecución de rutinas repetitivas en sistemas de tiempo real, el manejo inadecuado de la memoria RAM puede provocar la degradación del rendimiento o el colapso de la aplicación por desbordamiento de pila (*Stack Overflow*) (EEST N.º 1 "Eduardo Ader", 2026).

El propósito de esta actividad consiste en aplicar el Método 5R (Registrar, Reducir, Responder, Reflexionar y Revisar) para analizar el comportamiento del microprocesador frente a estructuras recursivas, la iteración sobre celdas contiguas de memoria y el direccionamiento indirecto por punteros, integrando estas técnicas dentro del flujo de desarrollo del proyecto *TeKnical May Cry (TmC)*.

2. Marco Teórico y Desarrollo Técnico

2.1. Recursividad y la Pila de Llamadas (*Stack*)

La recursividad es una técnica de programación donde una función se invoca a sí misma para resolver un problema dividiéndolo en instancias de menor complejidad. Cada llamada activa genera un marco de pila (*Stack Frame*) en el área de memoria *Stack*, reservando espacio para parámetros, variables locales y la dirección del registro de retorno. Para impedir una acumulación indefinida de marcos que agote la memoria del sistema operativo, resulta indispensable la inclusión de un **Caso Base**. Esta condición lógica interrumpe el flujo autorreferencial y da inicio a la fase de retorno de valores en la pila.

2.2. Arreglos Contiguos y Búsqueda Secuencial

Un arreglo en C++ asigna un bloque de direcciones físicas consecutivas en la memoria RAM. La distancia entre celdas adyacentes depende estrictamente del tamaño en bytes del tipo de dato declarado (4 bytes para variables enteras int). La búsqueda secuencial recorre el arreglo índice por índice para comparar el valor buscado. Para optimizar el tiempo de procesamiento en el algoritmo de complejidad temporal $O(n)$, se incluye la sentencia break, la cual interrumpe inmediatamente el bucle al hallar la primera coincidencia, evitando iteraciones innecesarias sobre el resto de las posiciones.

2.3. Direccionamiento Físico mediante Punteros

Un puntero (type*) es una variable de bajo nivel que almacena la dirección hexadecimal de memoria física de otra variable. Mediante el operador de dirección (&), se transmite la ubicación exacta en RAM a las funciones (pasaje por referencia/dirección). Posteriormente, el operador de desreferenciación (*) permite leer o escribir directamente sobre el valor alojado en dicha dirección sin generar copias secundarias en el *Stack*.

3. Código Fuente del Proyecto (src/main.cpp)

C++

```
/**
 * Materia: Laboratorio de Programación (LPR) - 5° 3°
 * Institución: E.E.S.T. N° 1 "Eduardo Ader" - Vicente López
 * Archivo: src/main.cpp
 * Actividad 6: Suite de Taller de Repaso e Integración
 * Estudiante: Ferrari, Ulises
 */

#include <iostream>
using namespace std;
```

```
// Prototipos de funciones modulares
int sumaRecursiva(int n);
void intercambiarValores(int* ptrA, int* ptrB);

int main() {
    cout << "=======================================" << endl;
    cout << " TALLER INTEGRADOR REPASO - ESTUDIANTE: Ferrari, Ulises" << endl;
    cout << "=======================================" << endl;

    // --- RETO 1: RECURSIVIDAD ---
    cout << "\n=== RETO 1: SUMA RECURSIVA ===" << endl;
    int num = 5;
    cout << "Suma acumulada de 1 hasta " << num << ": " << sumaRecursiva(num) << endl;

    // --- RETO 2: ARRAYS Y BÚSQUEDA SECUENCIAL ---
    cout << "\n=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===" << endl;
    const int TAM = 10;
    int vectorDatos[TAM] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
    int numeroBuscar = 18;
    bool hallado = false;

    for (int i = 0; i < TAM; i++) {
        if (vectorDatos[i] == numeroBuscar) {
            cout << "[SISTEMA] Valor " << numeroBuscar << " hallado en el indice [" << i << "]"
<< endl;
            cout << "Direccion RAM Hexadecimal: " << &vectorDatos[i] << endl;
            hallado = true;
            break; // Interrupción optimizada del ciclo O(n)
        }
    }

    if (!hallado) {
        cout << "[SISTEMA] El valor " << numeroBuscar << " no existe en el arreglo." << endl;
    }

    // --- RETO 3: INTERCAMBIO CON PUNTEROS ---
    cout << "\n=== RETO 3: INTERCAMBIO DE CELDAS ===" << endl;
    int x = 100, y = 500;
    cout << "Valores previos -> X: " << x << " | Y: " << y << endl;

    // Invocación pasando direcciones de memoria
    intercambiarValores(&x, &y);
    cout << "Valores actuales -> X: " << x << " | Y: " << y << endl;

    cout << "=======================================" << endl;
    return 0; // Finalización exitosa
}
```

```
// Desarrollo de la función recursiva con caso base garantizado
int sumaRecursiva(int n) {
    if (n <= 0) return 0; // Caso base obligatorio para prevenir Stack Overflow
    return n + sumaRecursiva(n - 1); // Llamada autorreferencial
}

// Desarrollo del intercambio físico mediante desreferenciación
void intercambiarValores(int* ptrA, int* ptrB) {
    int temporal = *ptrA;
    *ptrA = *ptrB;
    *ptrB = temporal;
}
```

Referencias Bibliográficas

EEST N.º 1 "Eduardo Ader". (2026). *Guía didáctica de Actividad 6: Taller práctico de repaso y consolidación en C++ - Laboratorio de Programación (5.° año)*. Dirección General de Cultura y Educación de la Provincia de Buenos Aires.

Ferrari, U. (2026). *Repositorio oficial del proyecto repasogeneral* [Repositorio de código fuente en GitHub]. GitHub. https://github.com/Ferraribip/repasogeneral.git